2.2 웹 페이지 구성

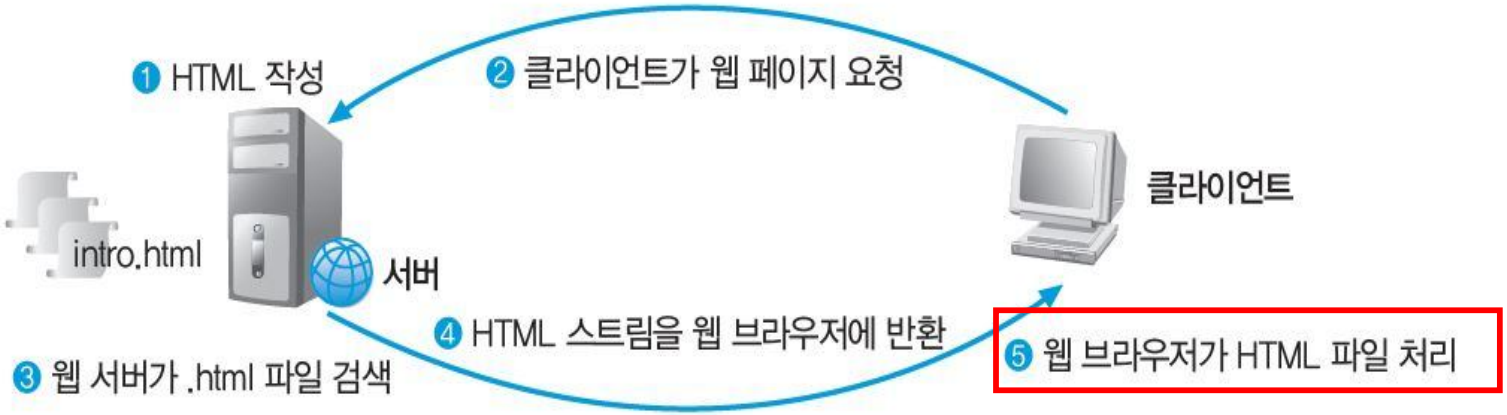
	구현 언어	구현 환경	실행 위치
정적 페이지 구성	HTML	Web Browser	클라이언트 측
동적 페이지 구성	JavaScript	JVM(**J**ava **V**irtual **M**achine)	클라이언트 측(CSS)
	ASP, JSP, PHP	CGI WAS	서버 측(SSS)

Contents 종류

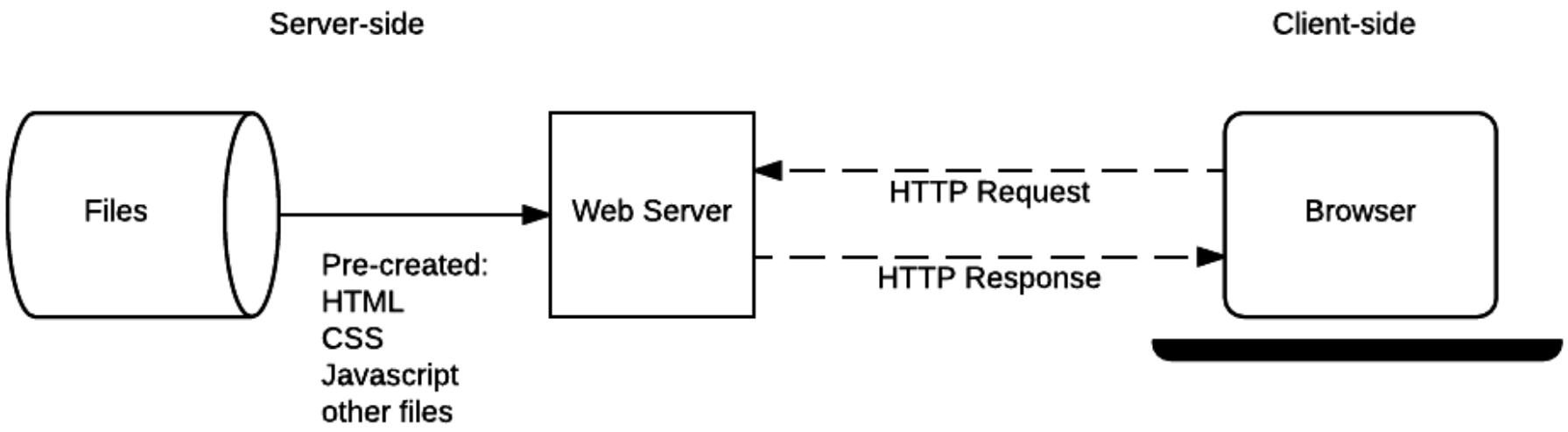
	설명	예제
정적 콘텐츠	• 이미지, 음악, 문서파일(hwp,doc)와 같이 웹페이지에 업로드되어 변경되지 않은 콘텐츠 • 웹 서버는 **정적**인 컨텐츠(html, css, js)를 제공하는 서버	확장자들이 jpg, png, hwp, docs, mp3 html, css, js 등을 가진 파일들
동적 콘텐츠	• 게시판과 같이 데이타베이스에 저장되어 수시로 변경되는 정보들 • WAS는 DB 조회나, 어떤 로직을 처리해야 하는 **동적**인 컨텐츠를 제공하는 서버	확장자가 php, asp, jsp등을 가진 파일들

1) HTML(Hyper Text Markup Language)

- 웹 서버에 HTML 문서를 저장하고 있다가 클라이언트가 요청하면 해당 페이지를 전송
- 클라이언트는 웹 페이지 해석 후 웹 브라우저로 나타냄
- 이를 정적인 웹 페이지라고 함.
- 정적인 웹 페이지는 변화를 반영하거나 새로운 것을 추가하기에 많은 시간이 걸림
- 웹 페이지는 바꿀 수 있는 가능성이 매우 낮다는 것이 보안상 장점

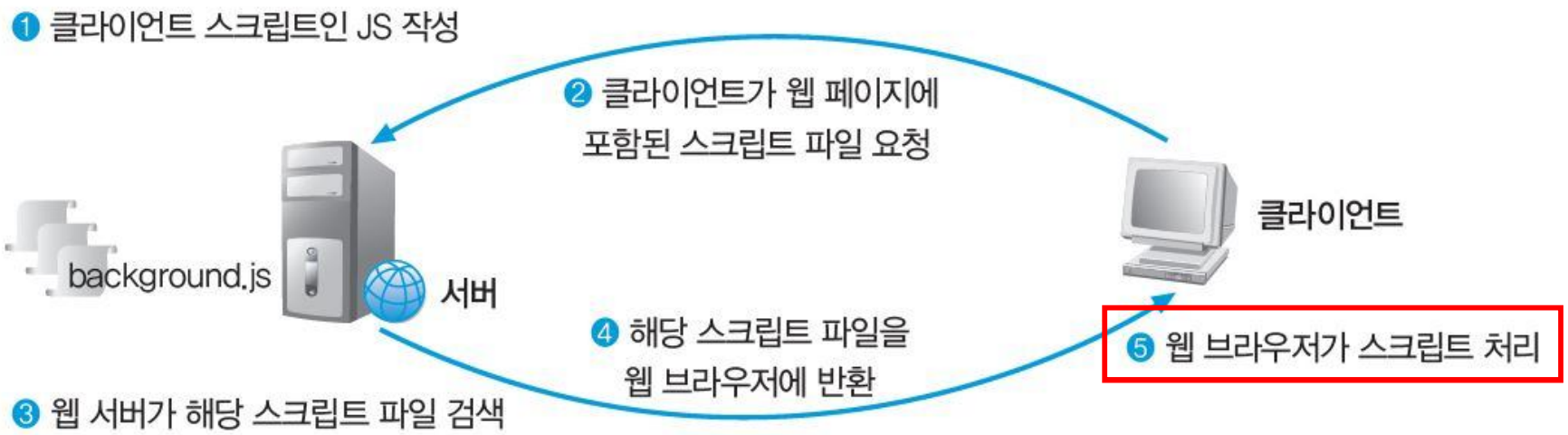


- Static Page (HTML)

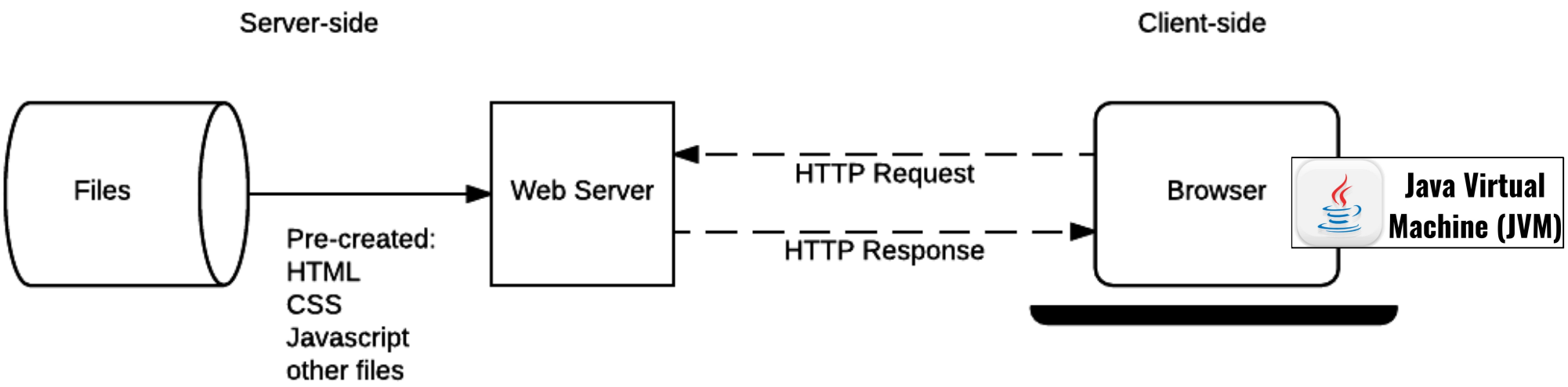


CSS(Client Side Script)

- 웹 서비스에 사용되는 스크립트, 클라이언트 측의 웹 브라우저에 의해 해석 및 적용
- 서버가 아닌 웹 브라우저에서 해석되어 화면에 적용
- 웹 서버의 부담을 줄이면서 다양한 기능을 수행할 수 있음
- JavaScript(객체지향언어), VB Script(주로 ASP와 연동)

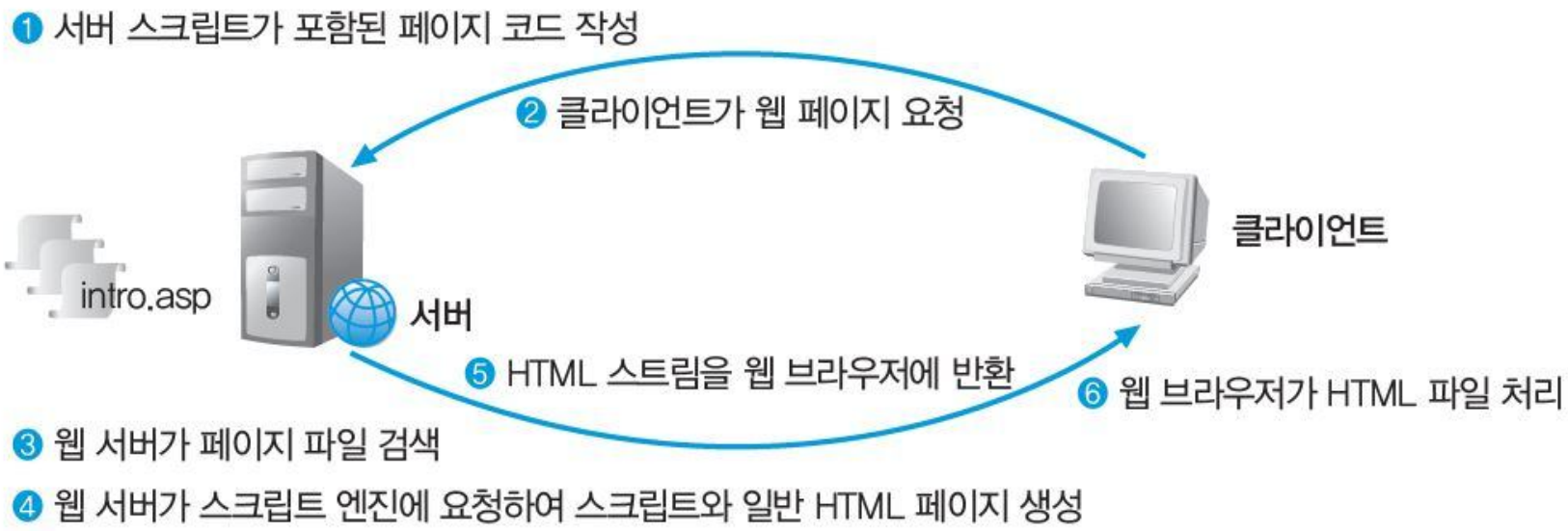


- Static sites (CSS)

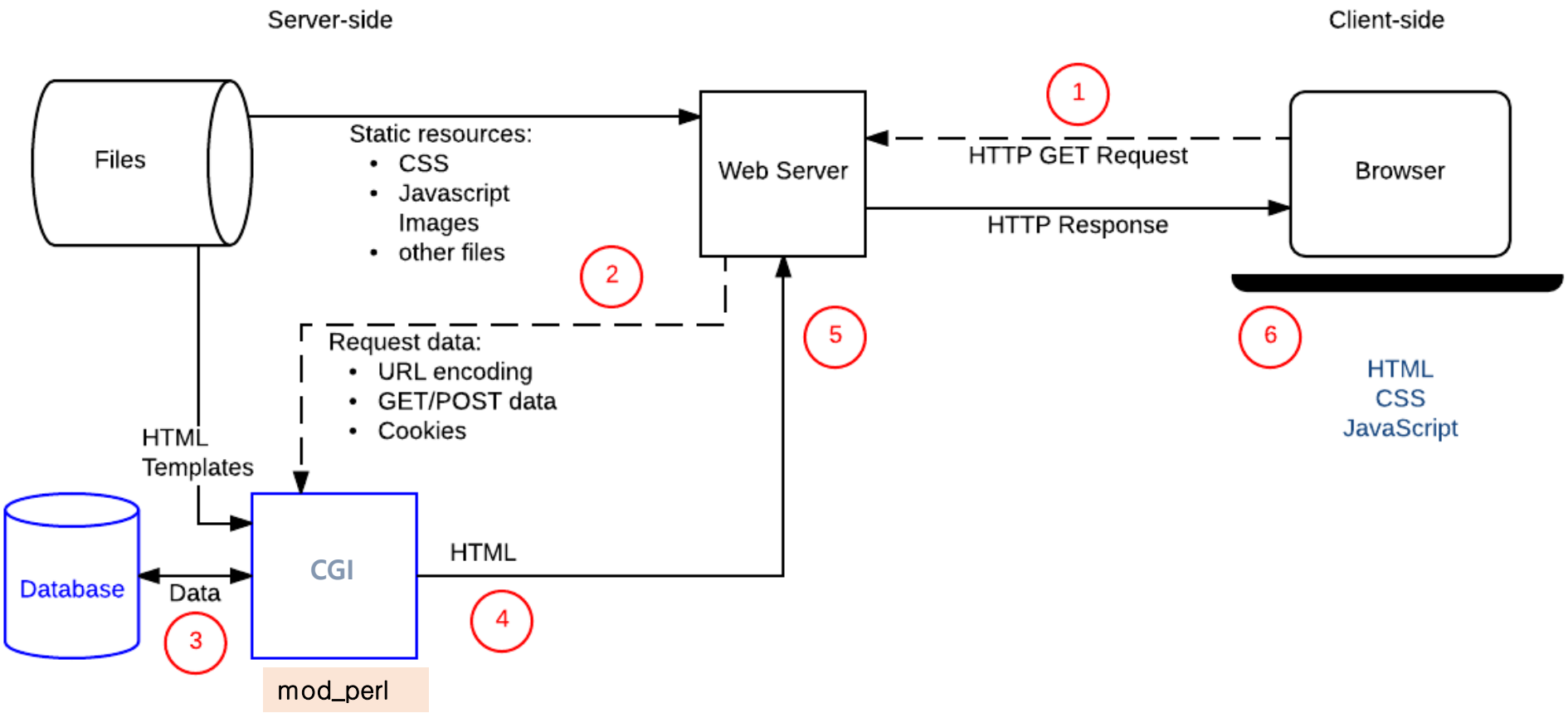


3) SSS(Server Side Script)

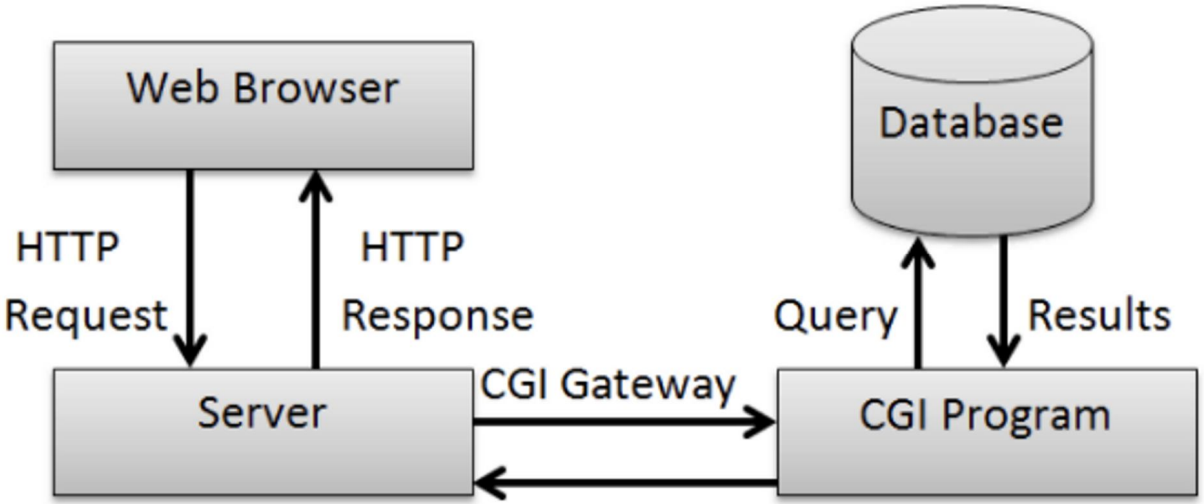
- CSS에서 받아온 정보를 처리하는 역할
- CSS에서 사용자가 어떠한 입력을 하게 되면 해당하는 정보를 SSS에서 처리하고 DB에 저장하거나 삭제
- 서버에 따라서 다른 언어를 사용하기 때문에 동작하는 웹 플랫폼이 다르기 때문에 맞춰서 사용



❶ CGI로 Dynamic sites 구현

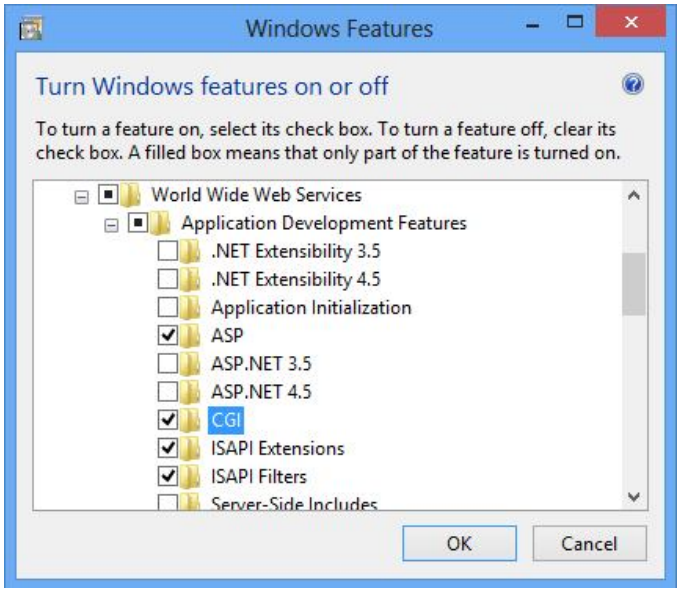


CGI(Common Gateway Interface)



- **웹 서버에서 프로그램을 동작시키기 위한 프로토콜**
- CGI는 웹 서버에 미리 저장된 HTML을 전달하는 것 뿐만 아니라, 사용자의 요청을 CGI 규격을 준수한 프로그램에서 적절하게 처리하고, 처리 결과를 HTML로 생성하여 웹 서버에 전달
- Python, Java, PHP 등의 프로그래밍 언어로 CGI 규격을 준수한 CGI Program을 만들면, 웹 서버는 CGI Program을 호출하고, 클라이언트의 요청에 대해 개별 프로세스를 생성

CGI interface 설치

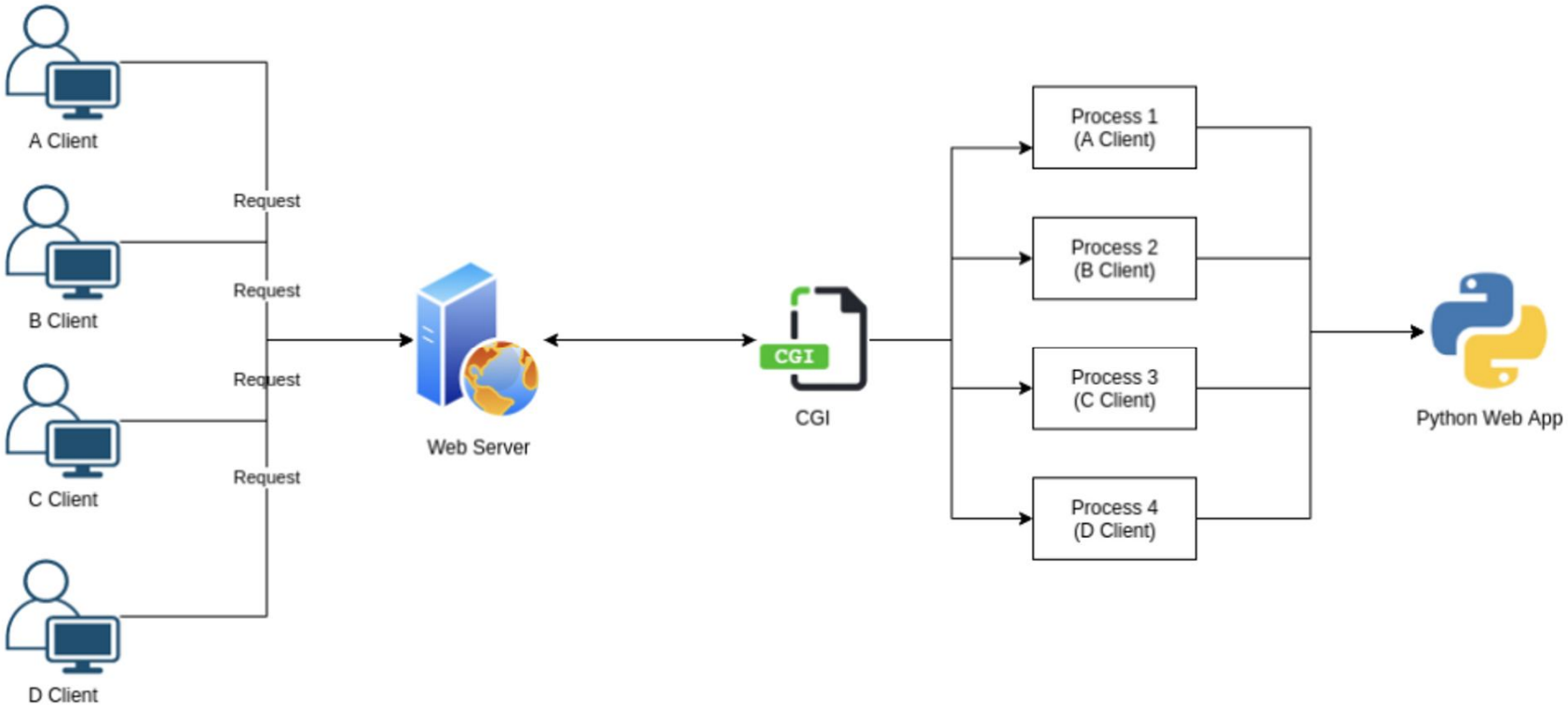


<< IIS 설치 시 >>

```
#cgiconfiguration
LoadModule cgi_module /usr/lib/apache2/modules/mod_cgi.so
ScriptAlias /cgi-bin/ /usr/local/apache2/cgi-bin/
<Directory /usr/local/apache2/cgi-bin>
        AllowOverride None
        AddHandler cgi-script .cgi .pl
        Options +ExecCGI -MultiViews +SymLinksIfOwnerMatch
        Order allow,deny
        Allow from all
        Require all granted
</Directory>
```

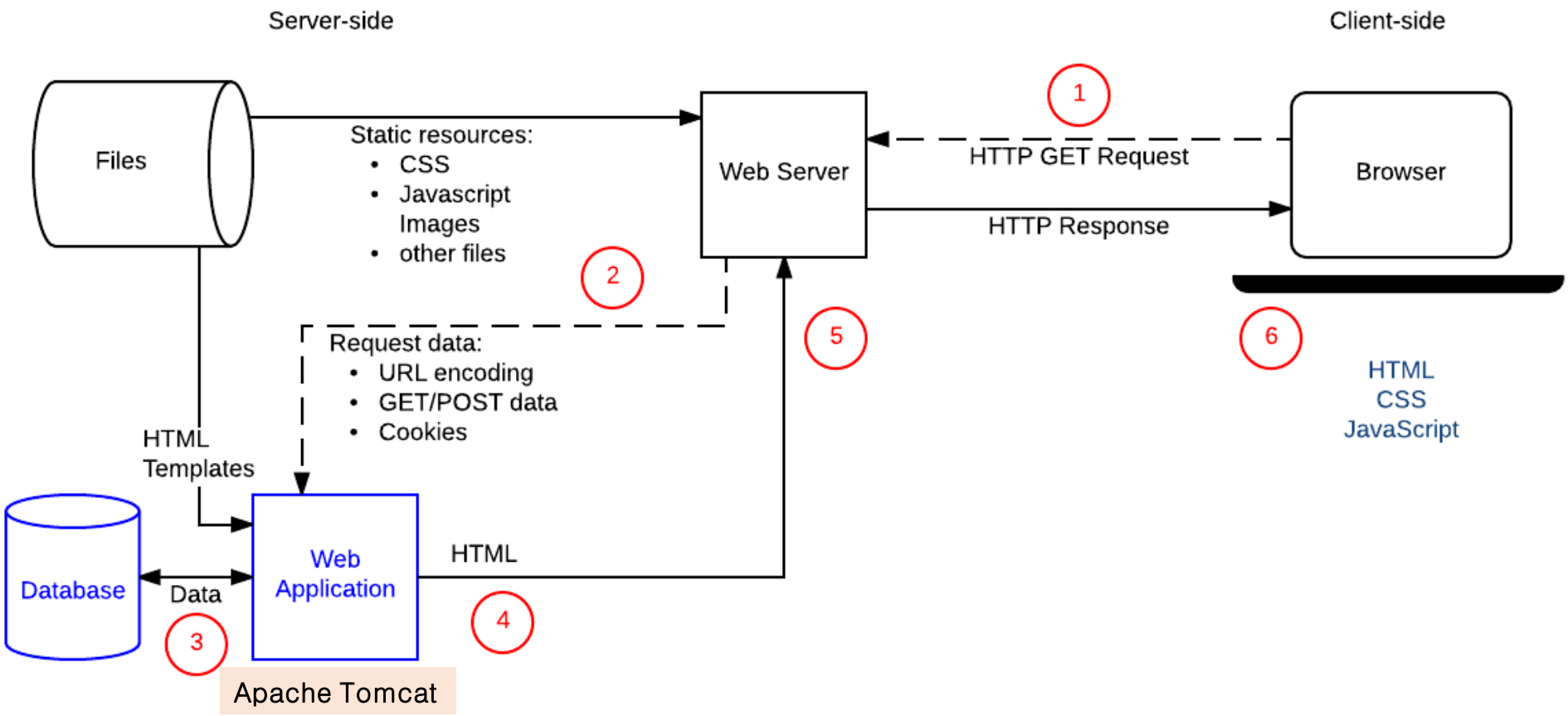
<< Apache 후 CGI 설정 >>

CGI(Common Gateway Interface) 한계

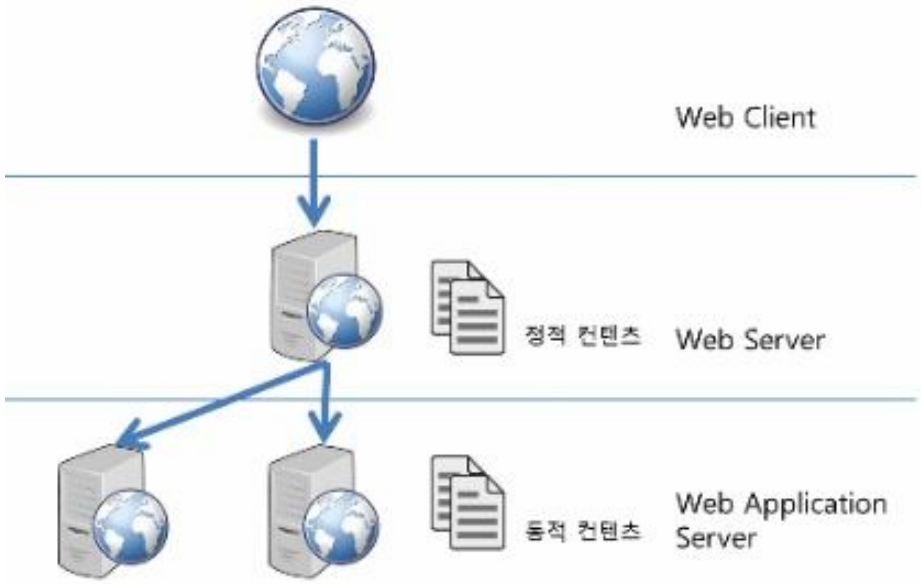


- 클라이언트의 요청이 많아지면 서버에 가해지는 부하를 적절하게 분배하지 못함
  - CGI가 클라이언트의 요청이 들어올 때마다 독립적인 프로세스를 생성하기 때문 (멀티 프로세싱)

❷ WAS로 Dynamic sites 구현



웹서버와 WAS를 분리하는 이유



WAS가 해야 할 일의 부담을 줄이기 위함

- 웹 서버에서는 정적인 문서만 처리, WAS는 애플리케이션의 로직만 수행하도록 기능을 분배
- WAS앞에 웹 서버를 둠으로써 서버의 부담을 줄일 수 있음
- 웹 서버에서는 플러그인 형태로 WAS를 연결하면 일 처리를 나눌 수 있음

WAS의 환경설정 파일을 외부에 노출시키지 않도록 하기 위함

- 클라이언트와 연결하는 포트가 직접 WAS에 연결이 되어 있다면 중요한 설정 파일들이 노출될 수 있음
- 웹 서버와 WAS에 접근하는 포트가 다르기 때문에, WAS에 들어오는 포트에는 방화벽을 둬어 보안을 강화할 수도 있음